

BACHARELADO EM INTELIGÊNCIA ARTIFICIAL

ÍTALO DIAS MOREIRA CAMPOS

PRÁTICA 01

ENTRADA ANALÓGICA COM ESP32

Relatório da disciplina Internet das Coisas e Robótica, referente à implementação física e aos testes de um sistema de leitura analógica com sinalização por LEDs.

Prof. Dr. Tassio Ferezini Martins Sirqueira

4º período | 2026

RESUMO

Este relatório descreve a montagem física e a programação de um sistema de entrada analógica com ESP32, uma placa com microcontrolador, isto é, um pequeno computador programável. Um potenciômetro de 10 kohms (dez quilohms, equivalentes a 10.000 ohms) foi utilizado como divisor de tensão, produzindo um sinal entre 0 V e 3,3 V no GPIO15. GPIO significa General Purpose Input/Output, ou pino de entrada e saída de uso geral. O conversor analógico-digital (ADC) de 12 bits transformou a tensão em números de 0 a 4095, utilizados para selecionar os estados apagado, verde, amarelo e vermelho. A montagem empregou GPIO5, GPIO18 e GPIO19 para os LEDs, resistores de 220 ohms e uma tela OLED de 128 x 64 pixels ligada por I2C. O programa gravado na placa (firmware) calculou a média de dez leituras para reduzir pequenas oscilações e aplicou histerese, isto é, limites ligeiramente diferentes na subida e na descida para evitar trocas rápidas de LED. Fotografias, Monitor Serial e vídeos confirmaram os quatro estados.

Palavras-chave: ESP32; entrada analógica; ADC; potenciômetro; histerese; OLED.

1 INTRODUÇÃO

Grandezas do mundo físico, como posição, temperatura e luminosidade, podem variar continuamente. Para que o ESP32 processe essas grandezas, o sinal analógico precisa ser digitalizado, ou seja, convertido em um número. Na Prática 01, essa ideia foi estudada por meio de um potenciômetro ligado a uma entrada analógica e de três LEDs usados como indicadores de faixa [1].

Além dos requisitos básicos, a montagem recebeu um display OLED e saída pelo Monitor Serial. Esses dois recursos facilitaram a verificação do sistema, pois permitiram comparar o valor do ADC, a tensão calculada e o estado do LED com o comportamento observado no circuito físico.

2 OBJETIVOS

2.1 Objetivo geral

Implementar e testar um sistema embarcado capaz de ler uma tensão analógica gerada por um potenciômetro e controlar três LEDs conforme a faixa medida.

2.2 Objetivos específicos

- Montar fisicamente o circuito em protoboard com alimentação de 3,3 V e GND comum.
- Ler o potenciômetro pelo GPIO15 e pelo ADC de 12 bits do ESP32.
- Converter a leitura de 0 a 4095 em uma tensão aproximada de 0 V a 3,3 V.
- Acionar somente o LED correspondente ao estado atual.
- Reduzir oscilações por média de amostras e histerese.

- Exibir os resultados no OLED e no Monitor Serial.
- Registrar a montagem e os testes por fotografias, capturas de tela e vídeo.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Potenciômetro e sinal analógico

O potenciômetro é um resistor variável com três terminais. A unidade de resistência elétrica é o ohm, representado normalmente pela letra grega ômega. O prefixo k significa mil; portanto, 10 kohms correspondem a 10.000 ohms. Quando os terminais laterais são conectados a 3,3 V e GND (referência elétrica de 0 V), o terminal central funciona como cursor de um divisor de tensão. Ao girar o eixo, a tensão central varia continuamente entre os dois extremos. Essa tensão foi aplicada ao GPIO15 e representou a entrada analógica do experimento [1].

3.2 Conversão analógico-digital

O ADC converte a tensão aplicada ao pino em um número inteiro. Um bit é a menor unidade de informação digital e pode valer 0 ou 1. Com 12 bits existem $2^{12} = 4096$ combinações possíveis, numeradas de 0 a 4095. A função `analogRead()` lê o pino analógico e retorna essa contagem. A documentação oficial do Arduino-ESP32 confirma que 12 bits correspondem à faixa de 0 a 4095 [2]. Na prática foi utilizada a expressão:

$$\text{tensão (V)} = \text{leitura ADC} \times 3,3 / 4095$$

Na expressão, V é a tensão estimada, ADC é o número lido e 3,3 V é a referência nominal da placa. A fórmula é didática e coerente com o roteiro. Entretanto, `analogRead()` retorna um valor bruto não calibrado; por isso, a tensão apresentada deve ser interpretada como estimativa, e não como medição exata [2].

3.3 LEDs e resistores

LED significa Light Emitting Diode, ou diodo emissor de luz. Ele possui polaridade: o ânodo é o terminal positivo e o cátodo é o terminal negativo. Cada LED foi ligado em série com um resistor de 220 ohms, isto é, no mesmo caminho da corrente, para limitar a corrente e proteger o componente e o ESP32.

3.4 Média e histerese

Ruído elétrico é uma pequena alteração indesejada no sinal. Para reduzir esse efeito, o firmware calcula a média de dez amostras, ou seja, realiza dez leituras, soma os valores e divide o total por dez. A histerese complementa esse tratamento ao usar um limite para entrar em um estado e outro para sair. A máquina de estados é a parte do programa que memoriza se o sistema está apagado, verde, amarelo ou vermelho. Assim, o estado anterior é mantido dentro de uma pequena margem e os LEDs não ficam alternando rapidamente.

3.5 OLED, I2C e relação com IoT

OLED significa Organic Light-Emitting Diode, tecnologia de tela baseada em diodos orgânicos emissores de luz. O módulo usado possui controlador SSD1306 e resolução de 128 x 64 pixels: 128 pontos na largura e 64 na altura; portanto, não se trata de "128C". A comunicação I2C (Inter-Integrated Circuit) utiliza SDA para transportar dados e SCL para sincronizar a comunicação. VCC fornece alimentação e GND é a referência elétrica. Os endereços 0x3C e 0x3D são números hexadecimais usados para identificar o display no barramento I2C.

IoT significa Internet of Things, ou Internet das Coisas. Este protótipo executa aquisição, processamento, decisão e atuação, mas ainda não envia dados pela Internet. Em uma evolução, os valores poderiam ser enviados por Wi-Fi usando MQTT, protocolo leve de mensagens, ou HTTP, protocolo utilizado por páginas e serviços web.

4 MATERIAIS E MÉTODOS

4.1 Materiais

- 1 placa ESP32 e cabo USB.
- 1 protoboard.
- 1 potenciômetro.
- 3 LEDs: verde, amarelo e vermelho.
- 3 resistores de 220 ohms.
- 1 display OLED 128 x 64 com interface I2C.
- Jumpers para as conexões.

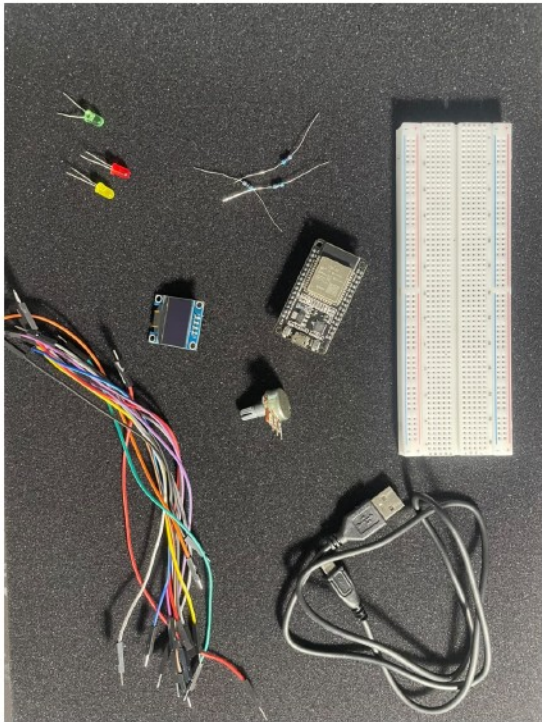


Figura 1 - Componentes utilizados na montagem física.

4.2 Pinagem e conexões elétricas

Componente	Pino	Conexão	Tensão	Função	Cuidado
Potenciômetro	Lateral	3V3	3,3 V	Alimentação	Não usar 5 V
Potenciômetro	Central	GPIO15	0-3,3 V	Sinal ADC	Respeitar a faixa
Potenciômetro	Lateral	GND	0 V	Referência	GND comum
LED verde	Ânodo	GPIO5 + 220 ohms	Lógico	Estado verde	Conferir polaridade
LED amarelo	Ânodo	GPIO18 + 220 ohms	Lógico	Estado amarelo	Conferir polaridade
LED vermelho	Ânodo	GPIO19 + 220 ohms	Lógico	Estado vermelho	Conferir polaridade
LEDs	Cátodo	GND	0 V	Retorno	Resistor em série
OLED	VDD/GND	3V3/GND	3,3/0 V	Alimentação	Conferir pinagem
OLED	SDA	GPIO21	3,3 V	Dados I2C	Não trocar com SCK
OLED	SCK	GPIO22	3,3 V	Clock I2C	SCL no código

Fluxo de energia: o cabo USB alimentou a placa, e os pinos 3V3 e GND distribuíram alimentação aos componentes. **Sinal elétrico:** o terminal central do potenciômetro produziu a tensão variável para o GPIO15. **Dados:** os pinos SDA e SCK transportaram a comunicação I2C entre o ESP32 e o OLED.

4.3 Montagem física

A montagem começou com o ESP32 sobre o canal central da protoboard. Em seguida foram distribuídos 3V3 e GND, instalado o potenciômetro, posicionados os LEDs e adicionados os resistores. Por último, os GPIOs e o OLED foram conectados.

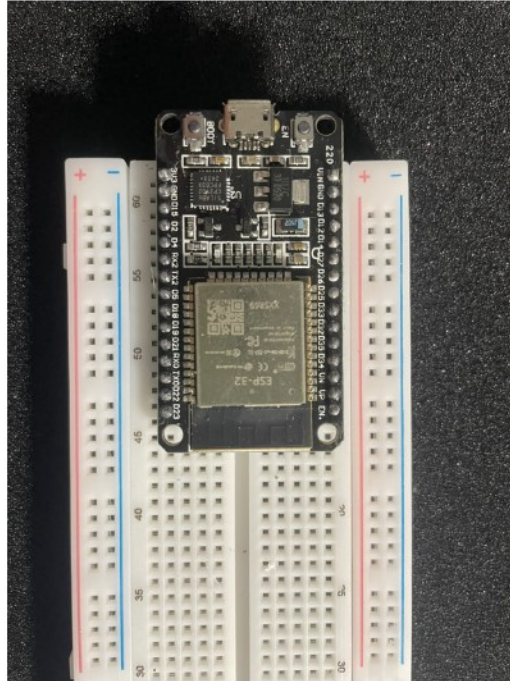


Figura 2 - ESP32 posicionado na protoboard.

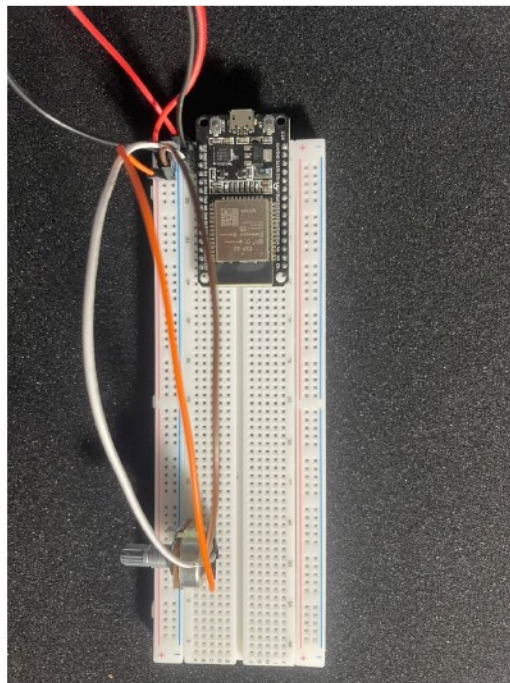


Figura 3 - Potenciômetro conectado ao GPIO15, 3V3 e GND.

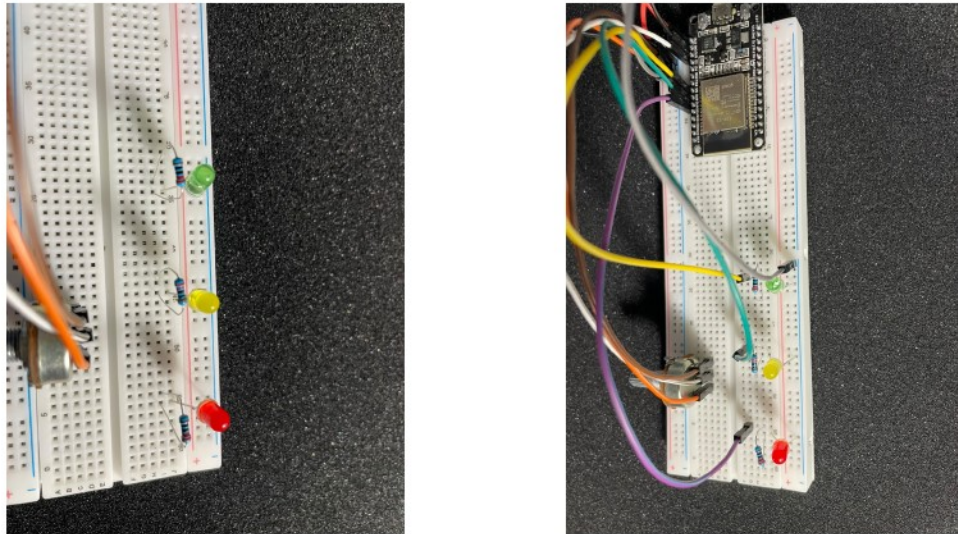


Figura 4 - Posicionamento dos resistores e conexões dos GPIOs dos LEDs.

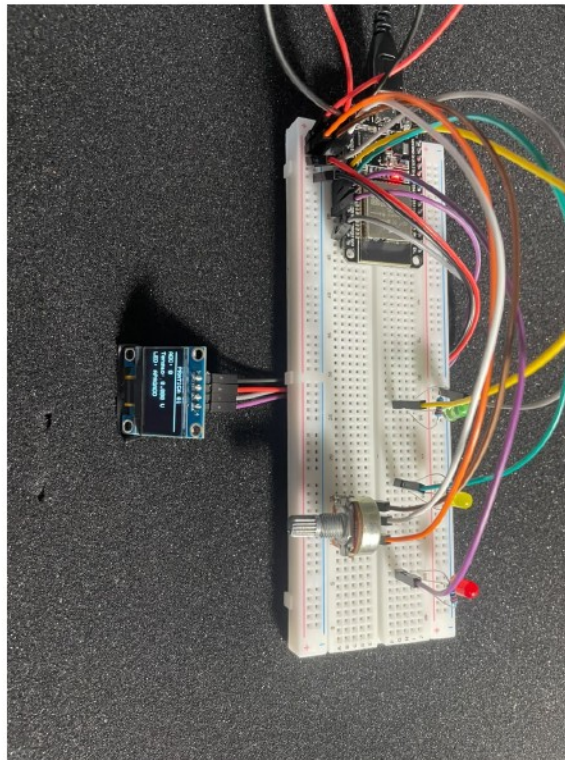


Figura 5 - Circuito completo com ESP32, potenciômetro, LEDs e OLED.

4.4 Firmware e procedimento de teste

O código completo está em `firmware/pratica01-entrada-analogica.ino`. Os nomes em português são identificadores válidos em C/C++ e foram escolhidos para facilitar o estudo. Os trechos seguintes reproduzem as partes mais importantes do programa realmente executado.

4.4.1 Mapeamento entre o código e a montagem

```
const int PINO_POTENCIOMETRO = 15;
const int PINO_LED_VERDE = 5;
const int PINO_LED_AMARELO = 18;
const int PINO_LED_VERMELHO = 19;
const int PINO_SDA = 21;
const int PINO_SCL = 22;
```

No código, `const int` declara um número inteiro que não muda durante a execução. Essas constantes ligam os nomes usados no programa aos pinos físicos. `PINO_POTENCIOMETRO` identifica a entrada; ele não controla sozinho todo o programa. O comportamento completo depende da leitura, da conversão, da decisão do estado e da atualização das saídas.

4.4.2 Média de 10 leituras e conversão

```
const int NUMERO_AMOSTRAS = 10;

int lerAdcComMedia() {
    long soma = 0;
    for (int i = 0; i < NUMERO_AMOSTRAS; i++) {
        soma += analogRead(PINO_POTENCIOMETRO);
        delay(2);
    }
    return soma / NUMERO_AMOSTRAS;
}
```

O laço `for` repete `analogRead()` dez vezes. `long` é um tipo de número inteiro com capacidade suficiente para acumular as leituras. A divisão final devolve a média, reduzindo pequenas oscilações sem inventar novos valores.

```
float converterAdcParaTensao(int valor_adc) {
    return (valor_adc * TENSÃO_REFERENCIA_V)
        / VALOR_MAXIMO_ADC;
}
```

`float` é um tipo numérico que aceita casas decimais. A função recebe a contagem do ADC e calcula a tensão estimada usando 3,3 V e 4095, exatamente como definido nas constantes do firmware.

4.4.3 Ciclo principal e inicialização

```
void loop() {
    leitura_adc = lerAdcComMedia();
    tensao_v = converterAdcParaTensao(leitura_adc);
    atualizarEstadoComHisterese();
    atualizarLeds();
    atualizarDisplay();
    atualizarSerial();
    delay(100);
}
```

A função `loop()` se repete continuamente enquanto o ESP32 permanece ligado. A ordem é: ler -> converter -> decidir -> acender o LED -> atualizar o OLED -> enviar os dados ao computador. `atualizarLeds()` primeiro apaga os três LEDs e depois acende apenas a cor correspondente, impedindo duas cores simultâneas.

A função `setup()` é executada uma única vez quando a placa inicia. Nela, o ADC é configurado em 12 bits; os LEDs são definidos como OUTPUT, ou modo de saída; o I2C é

iniciado nos GPIOs 21 e 22; e o OLED é procurado nos endereços 0x3C e 0x3D. Se o OLED não responder, os LEDs continuam funcionando.

4.5 Procedimento de teste

Na Arduino IDE foi selecionado ESP32 Dev Module, modelo de placa usado na compilação, e a porta COM correspondente, identificação da conexão USB no Windows. O firmware foi compilado, isto é, traduzido para o formato executado pelo ESP32, e enviado à placa. O Monitor Serial, janela que mostra as mensagens da placa, foi aberto em 115200 bit/s; bit/s significa bits transmitidos por segundo.

Os testes começaram com o potenciômetro no mínimo. O eixo foi girado lentamente para aumentar a tensão e registrar os estados apagado, verde, amarelo e vermelho. Depois, o movimento foi invertido para observar os limites de retorno da histerese. Para cada estado foram produzidas uma fotografia do circuito e uma captura do Monitor Serial. Também foram gravados vídeos do circuito e das transições.

5 RESULTADOS

5.1 Limites programados

Transição	Tensão aumentando	Tensão diminuindo
Apagado / verde	Verde acima de 0,55 V	Apagado abaixo de 0,45 V
Verde / amarelo	Amarelo acima de 1,55 V	Verde abaixo de 1,45 V
Amarelo / vermelho	Vermelho acima de 2,05 V	Amarelo abaixo de 1,95 V

5.2 Valores observados

Estado	ADC observado	Tensão observada	Confirmação física
Apagado	0	0,000 V	Todos os LEDs apagados
Verde	aprox. 663	aprox. 0,534 V	LED verde aceso
Amarelo	aprox. 1928	aprox. 1,554 V	LED amarelo aceso
Vermelho	2559	2,062 V	LED vermelho aceso

Os registros confirmaram a correspondência entre circuito, OLED e Monitor Serial. O valor de 0,534 V no estado verde não representa o instante exato de ativação. Como o estado já havia sido ativado acima de 0,55 V, a histerese o manteve ligado enquanto a tensão permaneceu acima de 0,45 V.

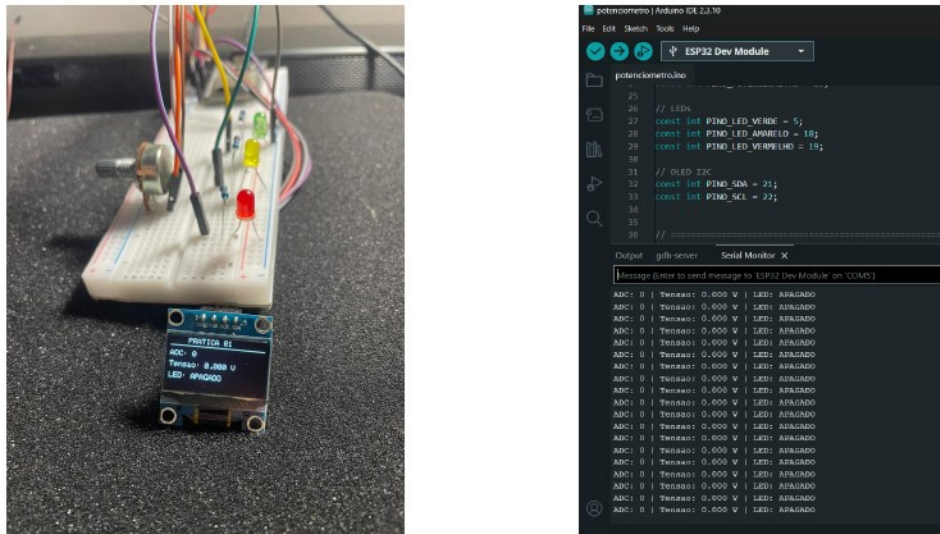


Figura 6 - Estado apagado: circuito físico e Monitor Serial em 0,000 V.



Figura 7 - Estado verde: circuito físico e Monitor Serial em aproximadamente 0,534 V.

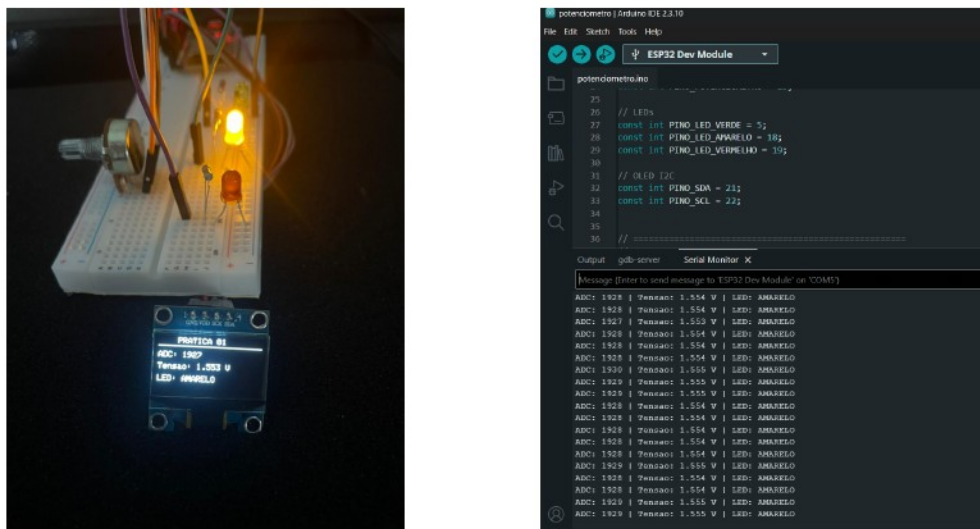


Figura 8 - Estado amarelo: circuito físico e Monitor Serial em aproximadamente 1,554 V.

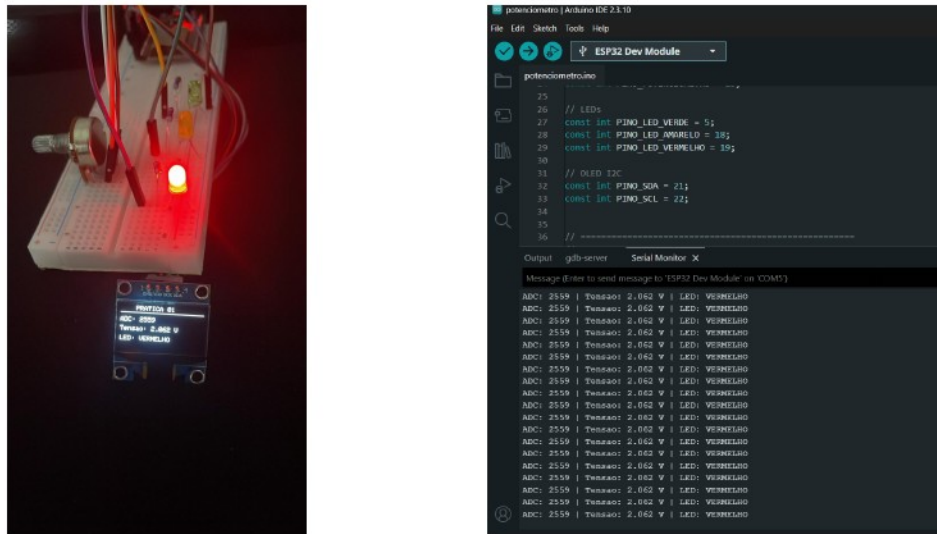


Figura 9 - Estado vermelho: circuito físico e Monitor Serial em 2,062 V.

5.3 Evidências complementares

No vídeo demonstracao-pratica-01.mp4, com aproximadamente 61 segundos, observa-se o giro físico do potenciômetro e a sequência apagado, verde, amarelo e vermelho. No vídeo teste-transicoes-monitor-serial.mp4, com aproximadamente 28,5 segundos, aparecem as leituras seriais próximas às trocas. Como o sistema trabalha por amostras, o primeiro número mostrado depois de uma transição pode ser um pouco maior que o limite programado. Os vídeos são evidências complementares; as fotografias e capturas mantêm os principais resultados visíveis no PDF.

6 AVALIAÇÃO DOS RESULTADOS

6.1 Qual a ordem de acionamento dos LEDs e em qual tensão cada LED acende?

Com a tensão aumentando, a ordem observada e programada é: todos apagados, LED verde, LED amarelo e LED vermelho. O verde entra acima de 0,55 V; o amarelo entra acima de 1,55 V; e o vermelho entra acima de 2,05 V. Os registros estáveis foram obtidos em 0,534 V para o verde, 1,554 V para o amarelo e 2,062 V para o vermelho. O primeiro valor ficou abaixo do limite de entrada porque a histerese manteve o estado verde após sua ativação.

6.2 Qual o papel do potenciômetro no circuito?

O potenciômetro funciona como um divisor de tensão ajustável. Seus terminais laterais foram ligados a 3,3 V e GND, e o terminal central forneceu ao GPIO15 uma tensão variável entre esses limites. Dessa forma, o giro mecânico simulou uma grandeza analógica que o ADC converteu em valor digital para a tomada de decisão do firmware.

7 DISCUSSÃO TÉCNICA

7.1 Decisões da montagem física

Na montagem física foram utilizados GPIO5, GPIO18 e GPIO19, que podem operar como saídas digitais [3], e resistores de 220 ohms em série com os LEDs. Esses são os componentes e pinos realmente usados e confirmados nos testes.

O OLED, a média de dez amostras e a histerese não eram indispensáveis para responder às questões da prática, mas melhoraram a observação e a estabilidade. O OLED permitiu acompanhar o sistema sem depender do computador; o Monitor Serial forneceu evidência numérica; e as fotografias comprovaram o acionamento físico. Essa combinação é mais confiável do que usar somente uma fonte de observação.

7.2 Limitações e possíveis melhorias

A principal limitação foi não usar um multímetro, aparelho que mede tensão elétrica, para conferir a tensão real no terminal central do potenciômetro. Por isso, valores como 0,534 V e 2,062 V são estimativas calculadas a partir do ADC. A média de dez leituras reduz pequenas oscilações, mas não substitui a comparação com uma medição real.

A conexão Wi-Fi e o envio de dados pela Internet também não foram testados. Portanto, o experimento é uma base para IoT, e não um sistema IoT completo. Como melhorias futuras, seria possível comparar as leituras com um multímetro e enviar os valores por MQTT ou HTTP. Essas melhorias não são falhas da prática atual; são apenas formas de ampliar o projeto.

8 CONCLUSÃO

A Prática 01 foi concluída com a montagem e o teste de um sistema de entrada analógica baseado em ESP32. O potenciômetro produziu uma tensão variável no GPIO15, o ADC converteu essa tensão em valores digitais e o firmware acionou corretamente os LEDs verde, amarelo e vermelho conforme as faixas definidas.

Os testes confirmaram os quatro estados e mostraram coerência entre LED, OLED e Monitor Serial. A adaptação dos GPIOs tornou a atividade compatível com o ESP32 físico, enquanto a média de dez leituras e a histerese tornaram o comportamento mais estável. Assim, os objetivos de compreender a entrada analógica, a conversão ADC e o controle de saídas digitais foram atendidos. A montagem também oferece uma base coerente para uma futura integração IoT, sem afirmar que a comunicação pela Internet já foi realizada.

REFERÊNCIAS

[1] ALGETEC. *Práticas de IoT: Entrada Analógica*. Apresentação, sumário teórico, roteiro e atividade da Prática 01. Material didático, s.d.

[2] ESPRESSIF SYSTEMS. *Arduino-ESP32: ADC API*. Disponível em: <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html>. Acesso em: 26 ago. 2026.

[3] ESPRESSIF SYSTEMS. *ESP-IDF Programming Guide: GPIO & RTC GPIO - ESP32*. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/gpio.html>. Acesso em: 26 ago. 2026.

SIRQUEIRA, Tassio Ferenzini Martins. *Internet das Coisas: aula 01*. Apresentação da disciplina, s.d.

FONTES DO EXPERIMENTO

CAMPOS, Ítalo Dias Moreira. *pratica01-entrada-analogica.ino: firmware da Prática 01*. Arquivo digital. 2026.

CAMPOS, Ítalo Dias Moreira. *Acervo fotográfico e vídeos da Prática 01*. Arquivos digitais. 2026.